

Assignment 3

1 Instructions

- This exercise must be implemented and submitted **at latest 24 hours before** your next laboratory class.
- Read the following documentation and, **first of all**, draw **UML Class Diagram** for both projects. Save them as image (*.png, *.jpg, *.jpeg) and **copy each one into the root of the corresponding project**.
- Rename projects (Refactor → Rename) as `surname1_surname2_name_lab03_dome` and `surname1_surname2_name_lab03_socialnetwork`, lowercase, without accents. Then, export them to a **single** ZIP file named `surname1_surname2_name_lab03` and upload it to the corresponding task in Campus Virtual.

Whatever the exercise, be sure you **always meet all the following rules**:

1. Unless other thing is stated, null are considered **invalid arguments** as well as empty (length 0) and blank (consists of whitespace) strings.
2. Write **tests first**.
3. When **updating** a project, check that existing methods keep working properly. Even more, include new characteristics in existing tests.
4. Review the **checklist** to check the correctness of your project according to the learning outcomes.
5. From now on, test folder must maintain exactly the same package structure than source folder. Then, JUnit cases will be named as `ClassNameMethodNameTests`.

2 Project Dome

First of all, complete the **UML class diagram** according to the following instructions.

2.1 Testing

First of all, read next section and **before implementing**, write functional and robustness tests for new public methods in `MediaLibrary`.

Write method `List<Post> getItems()` returning an `ArrayList` containing all items in the library. **It is not in the public interface** of class `MediaLibrary`, but it is a **package-private method**, useful when implementing tests.

2.2 New public methods in class MediaLibrary

Add the following methods to class MediaLibrary:

- `public Item searchItem(Item theItem)` Search the library for an item passed and returns the item found in the library. If it is not found, returns null. Think about going to a bookstore with a copy of a book and ask for this book. It can throw `IllegalArgumentException` when the argument is null.
- `public String getResponsibles()` Return a String with the *responsible* for all items owned, separated with commas. The responsible for CDs is the artist; for Dvd, the director. If there are not items owned, just return an empty String.

2.3 Add a new Item VideoGame.

A VideoGame has:

- `title`, `author`, `comment` and `gotIt`
- A number of players
- Platform, one of XBox, PlayStation or Nintendo
- **But** it does not have playing time
- There is a print method to show the videogame in the console
- The responsible is the author

2.4 Review previous classes and tests

Update class `MediaPlayer` so it also contains `VideoGame`.

Update also all tests so they also run with `VideoGame` and check they still pass in green.

3 The social network

The term social network refers to a computer-based technology that facilitates the sharing of ideas, thoughts, and information through virtual networks and communities.

A **post** is a piece of information published by a **user** on the network. They can be **liked** as a way to let people know that you enjoy the post. They can also be **commented** so a user replies to other users' post. And, they can also be **printed** out in a user-friendly way.

Each **post** can be one of the following two types: **text message** and **image**, with different content.

Text message As the name implies, its content is just a sequence of characters.

Image Such posts are made out of the image filename and the image caption.

The most popular activities on this social network are **adding posts**, **searching posts** and **list all posts** in the network

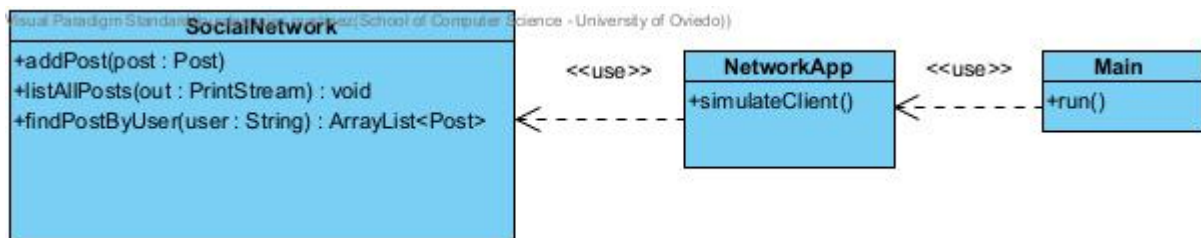
Add a post Add a post, any type, increasing the posts held. So, when adding a new post, the social network will contain every previous post plus the new one.

Find posts by user Return all posts (**if any**) by certain user.

List all posts Print a user-friendly version of all posts in the network.

3.1 UML

First of all, complete the following **UML class diagram**.



3.2 Testing

Write functional and robustness tests for all public methods in class **SocialNetwork**, except **listAllPosts**.

Here are a few suggestions to get you started:

- Write package-private method `List<Post> getPosts()` returning an `ArrayList` containing all posts in the social network. **It is not in the public interface** of class **SocialNetwork**, but could be useful when implementing tests.
- When writing tests for **addPost** check not only that the network **contains** the post added but **also all previous posts**. It may help you to use `getPosts()` and then, method `contains()` of class `java.util.ArrayList` that returns true if a list contains certain element.
- Finally, for testing `findPostsByUser()`, you might create your own `ArrayList` with the **expected posts** and then, use methods `contains` and `size` to check if the **expected** matches the value returned by `findPostsByUser()`.

3.3 Implement class SocialNetwork

Class **SocialNetwork** must **hold** and **manage** every post in the social network.

3.3.1 The posts

A post is a piece of information published by a user. Each **post** can be one of the following two types: **text message** and **image**.

Text message This type of posts only contains a **message** that gets published on the social network platform.

Image Such posts are made out of the image **filename** and a **caption**.

Regardless of the type, each post must contain

- the name of the **user** who created it,
- the number of **likes** on a post, and
- a list of **comments**.

Regardless the type, all posts have this behaviour

- You can write comments and like on any post.
- When a **new post is created**, arguments for every attribute must be provided except for likes, initially set to zero, and comments, initially empty.
- There must exist a method `public void print(PrintStream out)` that prints out a user-friendly string with the following format:
 - TEXT MESSAGE Posted by: the username, Content: the message
 - PHOTO Posted by: the username, File: the filename, Caption: the caption

3.3.2 Public interface of class SocialNetwork

This class must contain the following public methods.

`public void addPost(Post post)` Add to the network the post passed as long as it is not null; in that case, throws `IllegalArgumentException`

`public void listAllPosts(PrintStream out)` Print all posts, if any, each on a separate line.

Sample output:

PHOTO Posted by: mary1999, File: Cats.jpg, Caption: Funny cats

TEXT MESSAGE Posted by: Deskartessss, Content: I think therefore I am

`public List<Post> findPostsByUser(String user)` Return all posts by the user passed or empty if no post could be found by the user.

3.4 Other classes

Create a class `NetworkApp` with a single public method `public void simulateClient()`. First, create a `SocialNetwork` instance. Then, add several photos and texts from two different users, print all posts and, finally, get and print all posts associated with one user.

Finally, add also a `Main` class to run class `NetworkApp`.